

WHY DO GRAPHS EXIST?

Arrays and Trees represent linear or hierarchical data. But real-world data is a **network of connections** (social friends, web links, maps, dependencies).

Graphs model entities (**Vertices**) and relationships (**Edges**) with zero restrictions on cycles or connectivity.

THE VISITED LAW

A Tree is an acyclic graph. In Graphs, cycles exist! Traversal **MUST** track `visited[]` or `visited[][]` to prevent infinite recursion loop!

1 GRAPH TERMINOLOGY

- Vertex (V):** Node / entity in network
- Edge (E):** Connection / link between nodes
- Degree:** Count of edges attached to node
- In-Degree / Out-Degree:** Incoming vs outgoing edges
- Path / Cycle:** Sequence of nodes / closed loop

Type	Definition	Key Use Case
Undirected	2-way link (A ↔ B)	Social network friends
Directed	1-way arrow (A → B)	Twitter follower, Course prereq
Weighted	Edge has distance/cost	Google Maps, Flight costs
DAG	Directed Acyclic Graph	Build systems, Topological sort

2 VISUALIZING GRAPH TYPES

UNDIRECTED GRAPH

DIRECTED DAG

```
graph TD
classDef p fill:#faf5ff,stroke:#7c3aed,stroke-width:2px,color:#6b21a8,font-weight:700;
A((0)):::p-->B((1)):::p
A-->C((2)):::p
B-->C
```

Graph density: Sparse graph ($E \approx V$) vs Dense graph ($E \approx V^2$).

PREREQUISITES & COMPLEXITIES

Concept	Storage / Traversal	Complexity
Adjacency List	<code>List<Integer>[]</code>	Space: $O(V + E)$
Adjacency Matrix	<code>int[V][V]</code>	Space: $O(V^2)$
DFS Traversal	Stack / Call Stack	Time: $O(V + E)$
BFS Traversal	Queue (FIFO)	Time: $O(V + E)$

THE 4 GRAPH REPRESENTATION TYPES

- Edge List:** `int[][] edges` (raw input in 90% LC problems)
- Adjacency Matrix:** `int[][] grid` (lookup $O(1)$, space $O(V^2)$)
- Adjacency List:** `List<Integer>[]` (optimal for sparse graphs)
- Implicit Grid / Object Nodes:** 2D Matrix OR Node class with `neighbors`

1 CONVERSION: EDGE LIST → ADJACENCY LIST

Undirected Graph (2-way insertion):

```
List<Integer>[] adj = new ArrayList[n];
for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
for (int[] e : edges) {
    adj[e[0]].add(e[1]);
    adj[e[1]].add(e[0]); // 2-way link for undirected!
}
```

Directed Graph (1-way insertion):

```
for (int[] e : edges) {
    adj[e[0]].add(e[1]); // 1-way arrow e[0] → e[1]
}
```

2 CONVERSION: WEIGHTED EDGES → WEIGHTED ADJ LIST

Edges [u, v, weight] → List<int[]>[]:

```
List<int[]>[] adj = new ArrayList[n];
for (int i = 0; i < n; i++) adj[i] = new ArrayList<>();
for (int[] e : edges) {
    int u = e[0], v = e[1], w = e[2];
    adj[u].add(new int[]{v, w});
    adj[v].add(new int[]{u, w}); // Remove for directed
}
```

💡 Required for **Dijkstra's Algorithm** & **Prim's MST**!

3 CONVERSION: EDGE LIST → ADJACENCY MATRIX

```
int[][] matrix = new int[n][n];
for (int[] e : edges) {
    matrix[e[0]][e[1]] = 1;
    matrix[e[1]][e[0]] = 1; // Remove for directed
}
// For weighted graph: store weight instead of 1
```

O(1) edge existence check, but takes O(V²) memory.

4 GRID GRAPH MAPPING (4-Way & 8-Way Offsets)

```
// 4-Directional Vectors (Up, Down, Left, Right)
int[][] DIRS = {{-1,0}, {1,0}, {0,-1}, {0,1}};

// 2D Cell (r, c) to 1D Unique ID (for DSU):
int id = r * COLS + c;

// Reconstruct 2D (r, c) from 1D ID:
int r = id / COLS, c = id % COLS;
```

Bounds Guard: `r ≥ 0 && r < ROWS && c ≥ 0 && c < COLS`

TYPE 1

GRID GRAPH TRAVERSALS (2D Matrix DFS & BFS)

e.g. Number of Islands (LC 200), Max Area of Island (LC 695)

GRID DFS TEMPLATE

```
void dfs(char[][] grid, int r, int c) {
    if (r < 0 || r ≥ grid.length ||
        c < 0 || c ≥ grid[0].length ||
        grid[r][c] == '0') return;
    grid[r][c] = '0'; // mark visited
    in-place
    for (int[] d : DIRS)
        dfs(grid, r + d[0], c + d[1]);
}
```

GRID BFS TEMPLATE

```
void bfs(char[][] grid, int sr, int sc) {
    Queue<int[]> q = new LinkedList<>();
    q.offer(new int[]{sr, sc}); grid[sr][sc] = '0';
    while (!q.isEmpty()) {
        int[] curr = q.poll();
        for (int[] d : DIRS) {
            int nr = curr[0] + d[0], nc = curr[1] + d[1];
            if (nr ≥ 0 && nr < grid.length &&
                nc ≥ 0 && nc < grid[0].length &&
                grid[nr][nc] == '1') {
                grid[nr][nc] = '0'; // mark when enqueueing!
                q.offer(new int[]{nr, nc});
            }
        }
    }
}
```

KEY RULE

Mark `grid[nr][nc] = '0'` **BEFORE** adding to queue to avoid TLE/duplicate enqueueing!

Time: $O(R \times C)$

Space: $O(R \times C)$ worst case stack/queue

TYPE 2

TREE GRAPH TRAVERSALS (Acyclic Graph DFS & BFS)

e.g. Tree Diameter, Graph Valid Tree, Reconstruct Tree

PARENT-TRACK DFS (NO VISITED[] ARRAY NEEDED!)

```
void dfs(int u, int p, List<Integer>[] adj) {
    for (int v : adj[u]) {
        if (v ≠ p) { // Skip immediate parent to avoid backtrack!
            dfs(v, u, adj);
        }
    }
}
```

TREE BFS LEVEL ORDER

```
Queue<Integer> q = new LinkedList<>();
q.offer(root);
while (!q.isEmpty()) {
    int size = q.size();
    for (int i = 0; i < size; i++) {
        int u = q.poll();
        for (int v : adj[u]) q.offer(v);
    }
}
```

AHA INSIGHT

In a tree (acyclic graph), checking `v ≠ parent` replaces full `visited[]` tracking!

TYPE 3

ADJACENCY LIST TRAVERSALS (General Graph DFS & BFS)

e.g. Connected Components (LC 323), Path Exists (LC 1971)

GENERAL GRAPH DFS

```
void dfs(int u, List<Integer>[] adj, boolean[] vis) {
    vis[u] = true;
    for (int v : adj[u]) {
        if (!vis[v]) dfs(v, adj, vis);
    }
}
```

GENERAL GRAPH BFS

```
void bfs(int src, List<Integer>[] adj, boolean[] vis) {
    Queue<Integer> q = new LinkedList<>();
    q.offer(src); vis[src] = true;
    while (!q.isEmpty()) {
        int u = q.poll();
        for (int v : adj[u]) {
            if (!vis[v]) {
                vis[v] = true;
                q.offer(v);
            }
        }
    }
}
```

CONNECTED COMPONENTS

```
int count = 0;
for (int i = 0; i < n; i++) {
    if (!vis[i]) {
        dfs(i, adj, vis);
        count++;
    }
}
```

TYPE 4

OBJECT / CUSTOM NODE GRAPH TRAVERSALS (Clone Graph)

e.g. Clone Graph (LC 133)

CLONE GRAPH DFS WITH MAP VISITED

```
class Node { public int val; public List<Node> neighbors; }

Map<Node, Node> map = new HashMap<>();

Node cloneGraph(Node node) {
    if (node == null) return null;
    if (map.containsKey(node)) return map.get(node);

    Node copy = new Node(node.val, new ArrayList<>());
    map.put(node, copy);

    for (Node neighbor : node.neighbors) {
        copy.neighbors.add(cloneGraph(neighbor));
    }
    return copy;
}
```

CLONE GRAPH BFS TEMPLATE

```
Node cloneGraphBFS(Node node) {
    if (node == null) return null;
    Map<Node, Node> map = new HashMap<>();
    Queue<Node> q = new LinkedList<>();

    q.offer(node);
    map.put(node, new Node(node.val, new ArrayList<>()));

    while (!q.isEmpty()) {
        Node u = q.poll();
        for (Node v : u.neighbors) {
            if (!map.containsKey(v)) {
                map.put(v, new Node(v.val, new ArrayList<>()));
                q.offer(v);
            }
            map.get(u).neighbors.add(map.get(v));
        }
    }
    return map.get(node);
}
```

ADVANCED 1

MULTI-SOURCE BFS (Parallel Propagation)

e.g. Rotting Oranges (LC 994), 01 Matrix (LC 542), Walls and Gates

MULTI-SOURCE CONCEPT

- Enqueue ALL starting sources at t=0
- Process layer-by-layer (minute-by-minute)
- Computes minimum distance to ANY source

AHA

Instead of running BFS from each source separately (slow $O(k \cdot V)$), add all sources to queue initially and run ONE BFS (fast $O(V+E)$)!

TEMPLATE — ROTTING ORANGES (LC 994)

```
int orangesRotting(int[][] grid) {
    Queue<int[]> q = new LinkedList<>();
    int fresh = 0, time = 0;
    for (int r = 0; r < grid.length; r++)
        for (int c = 0; c < grid[0].length; c++)
            if (grid[r][c] == 2) q.offer(new int[]{r, c});
            else if (grid[r][c] == 1) fresh++;

    while (!q.isEmpty() && fresh > 0) {
        time++; int size = q.size();
        for (int i = 0; i < size; i++) {
            int[] curr = q.poll();
            for (int[] d : DIRS) {
                int nr = curr[0]+d[0], nc = curr[1]+d[1];
                if (nr >= 0 && nr < grid.length && nc >= 0 && nc < grid[0].length &&
                    grid[nr][nc] == 1) {
                    grid[nr][nc] = 2; fresh--; q.offer(new int[]{nr, nc});
                }
            }
        }
    }
    return fresh == 0 ? time : -1;
}
```

ADVANCED 2

TOPOLOGICAL SORT (Kahn's In-Degree BFS & Cycle Detection)

e.g. Course Schedule I & II (LC 207, 210), Alien Dictionary

KAHN'S ALGORITHM STEPS

1. Calculate `indegree[]` for all nodes
2. Push nodes with `indegree == 0` to Queue
3. Poll node, decrement neighbor indegrees
4. If neighbor indegree reaches 0 → enqueue
5. If processed count < V → **Cycle exists!**

TEMPLATE — COURSE SCHEDULE II (LC 210)

```
int[] findOrder(int n, int[][] prereqs) {
    List<Integer>[] adj = new ArrayList[n]; int[] in = new int[n];
    for (int i=0; i<n; i++) adj[i] = new ArrayList<>();
    for (int[] p : prereqs) { adj[p[1]].add(p[0]); in[p[0]]++; }

    Queue<Integer> q = new LinkedList<>();
    for (int i=0; i<n; i++) if (in[i] == 0) q.offer(i);

    int[] res = new int[n]; int idx = 0;
    while (!q.isEmpty()) {
        int u = q.poll(); res[idx++] = u;
        for (int v : adj[u]) if (--in[v] == 0) q.offer(v);
    }
    return idx == n ? res : new int[0]; // Cycle if idx != n
}
```

DSU

DISJOINT SET UNION (Union-Find with Path Compression & Rank)

e.g. Redundant Connection (LC 684), Number of Provinces (LC 547)

DSU OPERATIONS

- **Find(x):** Find root + Path Compression
- **Union(x, y):** Merge sets by Rank
- **Time:** $O(\alpha(N)) \approx O(1)$ amortized

TEMPLATE — DSU CLASS

```
class DSU {
    int[] parent, rank;
    DSU(int n) {
        parent = new int[n]; rank = new int[n];
        for (int i = 0; i < n; i++) parent[i] = i;
    }
    int find(int i) {
        if (parent[i] == i) return i;
        return parent[i] = find(parent[i]); // Path compression
    }
    boolean union(int i, int j) {
        int rootI = find(i), rootJ = find(j);
        if (rootI != rootJ) {
            if (rank[rootI] < rank[rootJ]) parent[rootI] = rootJ;
            else if (rank[rootI] > rank[rootJ]) parent[rootJ] = rootI;
            else { parent[rootJ] = rootI; rank[rootI]++; }
            return true;
        }
        return false; // Already in same set → Cycle detected!
    }
}
```

DIJKSTRA

DIJKSTRA'S SHORTEST PATH (Non-Negative Weighted Graph)

e.g. Network Delay Time (LC 743), Path with Min Effort

DIJKSTRA CONCEPT

Min Heap stores [cost, node].
Always expand nearest unvisited node.
Time: $O((V + E) \log V)$
Space: $O(V + E)$

TEMPLATE — NETWORK DELAY TIME (LC 743)

```
int networkDelayTime(int[][] times, int n, int k) {
    List<int[]>[] adj = new ArrayList[n+1];
    for (int i=1; i<=n; i++) adj[i] = new ArrayList<>();
    for (int[] t : times) adj[t[0]].add(new int[]{t[1], t[2]});

    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b)→a[0]-b[0]);
    int[] dist = new int[n+1]; Arrays.fill(dist, Integer.MAX_VALUE);
    pq.offer(new int[]{0, k}); dist[k] = 0;

    while (!pq.isEmpty()) {
        int[] cur = pq.poll(); int d = cur[0], u = cur[1];
        if (d > dist[u]) continue;
        for (int[] nei : adj[u]) {
            int v = nei[0], w = nei[1];
            if (dist[u] + w < dist[v]) { dist[v] = dist[u] + w; pq.offer(new int[]{dist[v], v}); }
        }
    }
    int max = 0; for (int i=1; i<=n; i++) { if (dist[i]==Integer.MAX_VALUE) return -1; max = Math.max(max, dist[i]); }
    return max;
}
```

MST

KRUSKAL'S MINIMUM SPANNING TREE (Sort Edges + DSU)

e.g. Min Cost to Connect All Points (LC 1584)

GRAPHS — WHICH ALGORITHM TO USE?

Start: What does the problem ask?

Connected components / island count

→

DFS / visited[]
BFS / [] matrix
DSU tracking

Shortest path in UNWEIGHTED graph

→

BFS (Queue) Guarantee shortest path

Shortest path in WEIGHTED graph

→

Dijkstra (PQ) Min Heap by distance

Parallel multi-source expansion

→

Multi-Source BFS Enqueue all sources at t=0

Course ordering / prerequisites

→

Topological Sort Kahn's BFS in-deg

Cycle detection in undirected graph

→

DSU / DFS Union returns false if cycle

Connect all points with min cost

→

Kruskal MST Sort edges + DSU

Grouping dynamic connections

→

Disjoint Set (DSU) Union-Find with rank

💡 Unweighted Shortest Path = BFS. Weighted Shortest Path = Dijkstra.

🔗 TRIGGER WORDS DIRECTORY

Trigger Word	Algorithm	Key Data Structure
islands, max area, matrix traversal	Grid DFS / BFS	visited[][] / DIRS
shortest path, fewest steps (unweighted)	BFS	Queue<Integer>
shortest path, minimum cost (weighted)	Dijkstra	PriorityQueue<int[]>
rotting, fire spread, distance to 0	Multi-Source BFS	Enqueue all sources
prerequisites, course schedule, alien order	Topo Sort	indegree[] + Queue
redundant edge, graph valid tree	DSU	parent[] + rank[]
connect all points, minimum cost tree	Kruskal MST	Edge sorting + DSU

👤 INTERVIEW TRADE-OFFS & FOLLOW-UPS

Interviewer Asks	Your Answer
Why BFS for shortest path, not DFS?	BFS explores level-by-level, ensuring first arrival at target is shortest. DFS can explore deep long paths first.
Why not Dijkstra for negative weights?	Dijkstra assumes distances are non-decreasing. Use Bellman-Ford for negative weights.
Matrix vs Adj List memory?	Adj List is $O(V+E)$ (optimal for sparse). Matrix is $O(V^2)$ (wasteful for sparse).

EASY — Grid & Basic Graph

LC 200 · Number of Islands

MEDIUM

Pattern: Grid DFS

Key Insight: Sink connected '1's to '0's.

Grid DFS

```
int numIslands(char[][] grid) {
    int count = 0;
    for (int r = 0; r < grid.length; r++)
        for (int c = 0; c < grid[0].length; c++)
            if (grid[r][c] == '1') { dfs(grid, r, c); count++; }
    return count;
}

void dfs(char[][] g, int r, int c) {
    if (r<0 || r>=g.length || c<0 || c>=g[0].length || g[r][c]=='0') return;
    g[r][c] = '0';
    for (int[] d : DIRS) dfs(g, r+d[0], c+d[1]);
}
```

LC 695 · Max Area of Island

MEDIUM

Pattern: Grid DFS Area

Key Insight: Return 1 + sum of child DFS calls.

Grid DFS

```
int maxAreaOfIsland(int[][] grid) {
    int max = 0;
    for (int r = 0; r < grid.length; r++)
        for (int c = 0; c < grid[0].length; c++)
            if (grid[r][c] == 1) max = Math.max(max, area(grid, r, c));
    return max;
}

int area(int[][] g, int r, int c) {
    if (r<0 || r>=g.length || c<0 || c>=g[0].length || g[r][c]==0) return 0;
    g[r][c] = 0; int a = 1;
    for (int[] d : DIRS) a += area(g, r+d[0], c+d[1]);
    return a;
}
```

LC 133 · Clone Graph

MEDIUM

Pattern: Object Graph DFS

Key Insight: HashMap tracks original → clone node.

Map Visited

```
Map<Node, Node> map = new HashMap<>();
Node cloneGraph(Node node) {
    if (node == null) return null;
    if (map.containsKey(node)) return map.get(node);
    Node copy = new Node(node.val, new ArrayList<>());
    map.put(node, copy);
    for (Node n : node.neighbors) copy.neighbors.add(cloneGraph(n));
    return copy;
}
```

MEDIUM — Multi-Source & Topo Sort

LC 994 · Rotting Oranges

MEDIUM

Pattern: Multi-Source BFS

Key Insight: Enqueue all 2s initially, step time.

Multi-Source BFS

```
int orangesRotting(int[][] grid) {
    Queue<int[]> q = new LinkedList<>();
    int fresh = 0, t = 0;
    for (int r=0; r<grid.length; r++)
        for (int c=0; c<grid[0].length; c++)
            if (grid[r][c] == 2) q.offer(new int[]{r, c});
            else if (grid[r][c] == 1) fresh++;
    while (!q.isEmpty() && fresh > 0) {
        t++; int sz = q.size();
        for (int i=0; i<sz; i++) {
            int[] cur = q.poll();
            for (int[] d : DIRS) {
                int nr=cur[0]+d[0], nc=cur[1]+d[1];
                if (nr>=0 && nr<grid.length && nc>=0 && nc<grid[0].length && grid[nr][nc]==1) {
                    grid[nr][nc]=2; fresh--; q.offer(new int[]{nr, nc});
                }
            }
        }
    }
    return fresh == 0 ? t : -1;
}
```

LC 207 · Course Schedule I

MEDIUM

Pattern: Topological Cycle Detect

Key Insight: Kahn's BFS. Processed count == n.

Kahn's BFS

```
boolean canFinish(int n, int[][] prereqs) {
    List<Integer>[] adj = new ArrayList[n];
    int[] in = new int[n];
    for (int i=0; i<n; i++) adj[i] = new ArrayList<>();
    for (int[] p : prereqs) { adj[p[1]].add(p[0]); in[p[0]]++; }
    Queue<Integer> q = new LinkedList<>();
    for (int i=0; i<n; i++) if (in[i] == 0) q.offer(i);
    int count = 0;
    while (!q.isEmpty()) {
        int u = q.poll(); count++;
        for (int v : adj[u]) if (--in[v] == 0) q.offer(v);
    }
    return count == n;
}
```


MEDIUM & HARD — DSU & Shortest Path

LC 684 · Redundant Connection

MEDIUM

Pattern: DSU Cycle Detect

Key Insight: Union edges. Edge connecting same root is redundant!

DSU

```
int[] findRedundantConnection(int[][] edges) {
    DSU dsu = new DSU(edges.length + 1);
    for (int[] e : edges) {
        if (!dsu.union(e[0], e[1])) return e;
    }
    return new int[0];
}
```

LC 743 · Network Delay Time

MEDIUM

Pattern: Dijkstra Shortest Path

Key Insight: Min Heap by distance. Max dist of all nodes.

Dijkstra PQ

```
int networkDelayTime(int[][] times, int n, int k) {
    List<int[]>[] adj = new ArrayList[n+1];
    for (int i=1; i<=n; i++) adj[i] = new ArrayList<>();
    for (int[] t : times) adj[t[0]].add(new int[]{t[1], t[2]});
    PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[0]-b[0]);
    int[] dist = new int[n+1]; Arrays.fill(dist, Integer.MAX_VALUE);
    pq.offer(new int[]{0, k}); dist[k] = 0;
    while (!pq.isEmpty()) {
        int[] cur = pq.poll(); int d = cur[0], u = cur[1];
        if (d > dist[u]) continue;
        for (int[] nei : adj[u]) {
            if (dist[u] + nei[1] < dist[nei[0]]) {
                dist[nei[0]] = dist[u] + nei[1];
                pq.offer(new int[]{dist[nei[0]], nei[0]});
            }
        }
    }
    int max = 0; for (int i=1; i<=n; i++) {
        if (dist[i]==Integer.MAX_VALUE) return -1;
        max = Math.max(max, dist[i]);
    }
    return max;
}
```

HARD & EXPERT — MST & Alien Order

LC 1584 · Min Cost to Connect All Points

MEDIUM

Pattern: Kruskal MST

Key Insight: Manhattan distance edges → Sort + DSU.

Kruskal DSU

```
int minCostConnectPoints(int[][] pts) {
    int n = pts.length; List<int[]> edges = new ArrayList<>();
    for (int i=0; i<n; i++)
        for (int j=i+1; j<n; j++)
            edges.add(new int[]{Math.abs(pts[i][0]-pts[j][0])+Math.abs(pts[i][1]-pts[j][1]), i, j});
    edges.sort((a,b) -> a[0]-b[0]);
    DSU dsu = new DSU(n); int cost = 0, count = 0;
    for (int[] e : edges) {
        if (dsu.union(e[1], e[2])) { cost += e[0]; if (++count == n-1) break; }
    }
    return cost;
}
```

LC 269 · Alien Dictionary

HARD

Pattern: Hard Topo Sort

Key Insight: Compare adjacent words for character ordering DAG.

Kahn's BFS

```
// Compare words[i] & words[i+1] → first differing char creates edge
// Run Kahn's Topological Sort on Character Graph
// Return empty string if cycle detected or invalid prefix!
```

COMPLETE GRAPH PROBLEM LADDER SUMMARY			
LC #	Problem	Pattern	Diff
200	Number of Islands	Grid DFS	M
695	Max Area of Island	Grid DFS Area	M
133	Clone Graph	Object Graph DFS	M
994	Rotting Oranges	Multi-Source BFS	M
207	Course Schedule I	Topo Sort Cycle	M
210	Course Schedule II	Kahn's BFS Order	M
684	Redundant Connection	DSU Cycle	M
743	Network Delay Time	Dijkstra Shortest	M
1584	Min Cost Connect Points	Kruskal MST	M
269	Alien Dictionary	Hard Topo Sort	H

🔍 DRY RUN — Dijkstra Min-Heap Shortest Path

Source K=1, Edges: (1→2, w=1), (1→3, w=4), (2→3, w=1)

Step	PQ Pop (dist, node)	Dist Array	Action / Relaxation
1	(0, 1)	[0, 0, ∞, ∞]	Check neighbors 2 (cost 1) & 3 (cost 4) → Push (1,2), (4,3)
2	(1, 2)	[0, 0, 1, 4]	Check neighbor 3 (1+1=2 < 4) → Relax dist[3]=2! Push (2,3)
3	(2, 3)	[0, 0, 1, 2]	Check neighbors → None. Processed!
4	(4, 3)	[0, 0, 1, 2]	4 > dist[3]=2 → Skip stale entry! ✅

✅ MASTERY CHECKLIST — CAN YOU DO THIS?

- ☐ Build Adj List from Edge List in 10s
- ☐ Write Grid DFS with bounds check from memory
- ☐ Write Grid BFS with enqueueing visited mark
- ☐ Write Multi-Source BFS (Rotting Oranges)
- ☐ Code Kahn's Topological Sort BFS from memory
- ☐ Implement DSU with Path Compression
- ☐ Code Dijkstra Shortest Path with PQ
- ☐ Code Kruskal MST with Edge sorting + DSU
- ☐ Identify Graph pattern from problem clues
- ☐ Solve LC 200, 994, 207, 743 within 20m each

📐 MATHEMATICAL COMPLEXITY DERIVATIONS

Why DFS/BFS is $O(V + E)$:
Every node is enqueued/visited at most once ($O(V)$). Every edge is traversed at most once in directed (twice in undirected) ($O(E)$). Total = $O(V + E)$.

Why Dijkstra is $O((V + E) \log V)$:
Min-heap holds at most E edges. Each edge relaxation does a `pq.offer()` of $O(\log V)$. Total = $O(E \log V + V \log V)$.

📦 GOLDEN RULES — NEVER FORGET

- Rule 1 — Mark Visited on Enqueue**

In BFS, set `visited = true` immediately when pushing to Queue, NOT when polling!
- Rule 2 — Unweighted Shortest Path = BFS**

BFS guarantees shortest path in unweighted graphs. Dijkstra is ONLY for weighted graphs!
- Rule 3 — Skip Stale Dijkstra Entries**

Always check `if (d > dist[u]) continue;` when polling from Dijkstra PriorityQueue!